

Synopsis

On

SENTINEL – INTELLIGENT THREAT DETECTION AND LOG ANALYSIS SYSTEM FOR IOT DEVICES



Group ID: SY402

Team Details

Enrolment No.	Name of Student	Class
ADT25SOCB1046	Saksham Jadhav	SY 4
ADT25SOCB1147	Satish Devde	SY 13
ADT25SOCB0955	Ritesh Gholap	SY 14
ADT25SOCB0056	Aditi Thakare	SY 8

Under the Guidance of

Prof. Revati Raspayle

Department of Computer Science & Engineering

MIT School of Computing

Sem – 3 | A. Y. 2026–2027

1. Project Title

Sentinel – Intelligent Threat Detection and Log Analysis System for IoT Devices

2. Abstract

IoT devices such as industrial sensors, smart cameras, and smart locks continuously generate system, authentication, and network logs. As the number of connected devices grows, manually monitoring these logs becomes difficult, and suspicious activity can go unnoticed.

Sentinel is a proposed threat detection and log analysis system for IoT environments. It collects relevant IoT logs and analyzes them using Rule-Based Detection for known attack patterns and Machine Learning for anomaly detection, so that both known and previously unseen abnormal behavior can be flagged for review.

Detected events are presented on a centralized dashboard showing the affected device, threat category, severity level, relevant evidence, and available source information such as an IP address when present in the logs. The system also suggests recommended response actions to assist an administrator — it does not take those actions automatically.

3. Introduction

The growing use of IoT devices across industrial, commercial, and smart environments has led to large volumes of system, authentication, and network log data. This data contains useful signals about device behavior, but reviewing it manually does not scale.

Activities such as repeated failed logins, port scanning, abnormal traffic patterns, unauthorized access attempts, and configuration changes may go unnoticed without automated support. Sentinel is proposed as a centralized system that collects and analyzes IoT logs, combines rule-based detection with machine learning, and presents the results through a single dashboard for administrators.

4. Problem Definition

IoT environments generate large volumes of system, authentication, and network logs, making manual monitoring difficult. Suspicious activities such as repeated failed login attempts, port scanning, abnormal network traffic, unauthorized access, and configuration changes may remain unnoticed. There is therefore a need for a centralized system that can automatically collect and analyze IoT logs, detect potentially suspicious behavior, identify affected devices, and present meaningful security information to administrators.

5. Objectives

- Develop a threat detection and log analysis system for IoT environments.
- Collect relevant system, authentication, and network logs from IoT devices.
- Process and centralize collected logs for analysis.
- Detect known suspicious activities using Rule-Based Detection.
- Identify abnormal behavior using Machine Learning-based anomaly detection.
- Identify the IoT device potentially affected by a detected event.
- Classify detected events by threat category and assign a severity level.
- Display relevant evidence and available source information, such as an IP address, when present in the logs.

- Present results through a centralized security dashboard.
- Generate alerts and recommended response actions for administrators.

6. Scope and Limitations

In scope for the current (initial) version

- Log collection from IoT devices or a simulated IoT environment.
- Log processing and normalization.
- Rule-based detection for known attack patterns.
- ML-based anomaly detection for abnormal behavior.
- Device and activity tracking.
- Threat classification, severity assignment, and visualization.
- Alert generation and recommended (not automated) response actions.

Explicitly out of scope for the current version

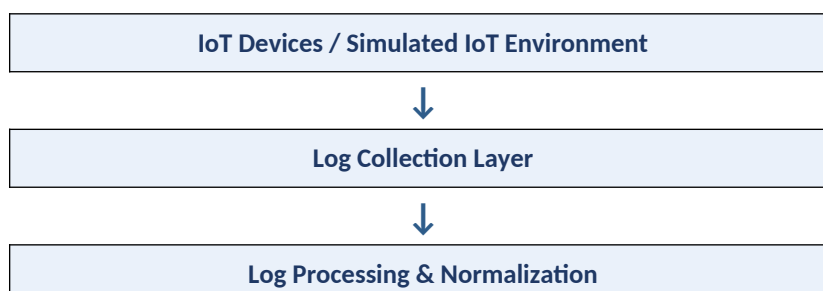
- Not an antivirus or endpoint protection product.
- Not a firewall or network traffic blocker.
- Not a complete enterprise Security Operations Center (SOC).
- Not a full SIEM replacement.
- Not an enterprise-scale DDoS mitigation platform — Sentinel focuses on detecting and reporting DDoS-like patterns, not absorbing or mitigating large-scale attack traffic.
- Not a penetration testing framework or password cracking tool.
- Not a complete automated incident response platform — response actions are recommended to a human administrator, not executed automatically.
- Does not claim to prevent every attack or determine the real-world identity of an attacker.

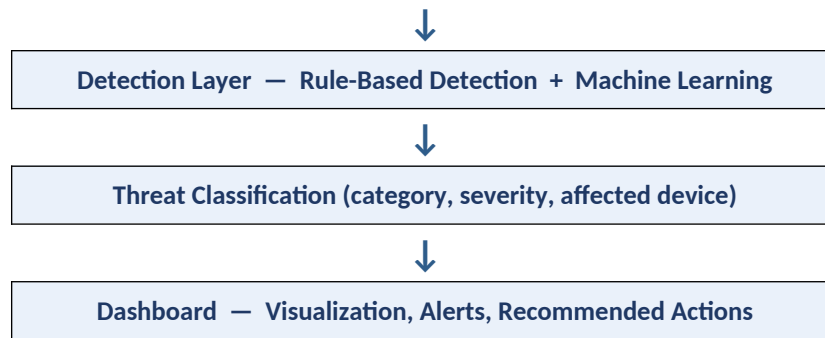
7. Proposed System

Sentinel collects relevant logs generated by connected IoT devices and processes them for security analysis. Rule-Based Detection identifies known suspicious patterns, while Machine Learning helps identify abnormal behavior that may not exactly match a predefined rule. Detected events are analyzed and presented through a centralized dashboard containing the affected device, threat category, severity, relevant evidence, and available source information. Sentinel also provides recommended actions to assist administrators in responding to detected incidents.

8. System Architecture

At a high level, Sentinel is organized into a log collection layer, a detection layer (rule-based + ML), and a presentation layer (dashboard and alerts), as shown below.





9. Four Main Modules

The project is presented as four main modules. Each detailed component described later in this document belongs to exactly one of these four modules, so the technical breakdown and the presentation structure remain consistent.

#	Main Module	Purpose
1	Log Collection and Monitoring	Collects and prepares raw IoT logs for analysis.
2	Threat Detection and Analysis	Applies rule-based detection and ML to identify suspicious events.
3	Device and Activity Tracking	Links detected events to the affected device and its activity history.
4	Dashboard, Alerts and Response	Visualizes results and issues alerts with recommended actions.

10. Detailed Internal Components

Module 1 — Log Collection and Monitoring

- IoT Log Collection — gathers system, authentication, and network logs.
- Log Processing and Analysis — parses, filters, and normalizes logs for downstream use.

Module 2 — Threat Detection and Analysis

- Rule-Based Detection — matches logs against known suspicious patterns.
- Machine Learning — flags behavior that deviates from learned normal patterns.
- Threat Detection — combines both outputs into a single detection result.

Module 3 — Device and Activity Tracking

- Device and Activity Tracking — identifies the affected device and its recent activity.

Module 4 — Dashboard, Alerts and Response

- Dashboard and Visualization — presents detected events centrally.
- Alert and Response — generates alerts and recommended (human-executed) response actions.

11. Threat Examples

The following are illustrative examples the system is designed to help detect. They are examples for design and testing purposes, not a claim of complete attack coverage.

- Brute Force Attack
- DoS Attack
- DDoS Activity (detection and reporting only, not mitigation)
- Botnet Activity
- Port Scanning
- Unauthorized Login
- Configuration Tampering
- Firmware Tampering
- Abnormal Network Behavior
- Suspicious IP Communication

12. Rule-Based Detection

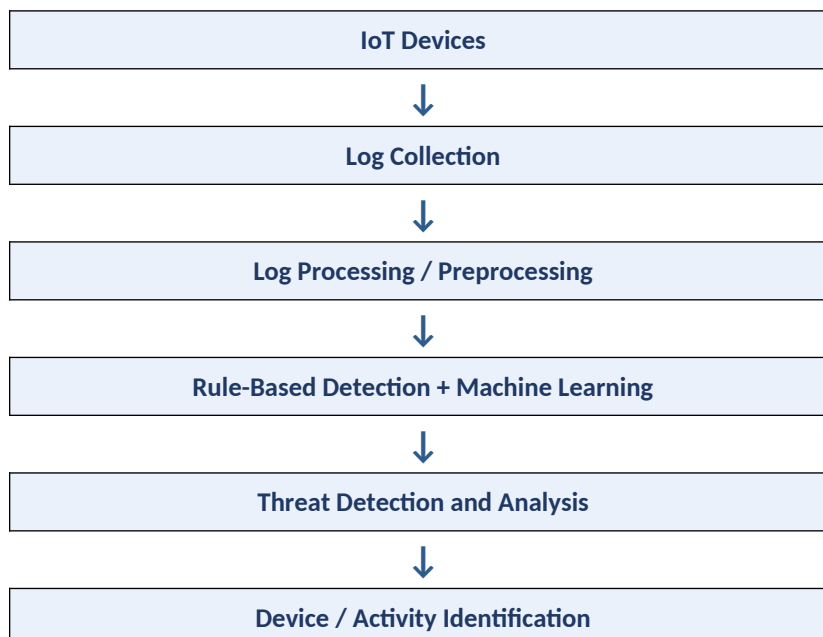
Rule-Based Detection is used for known, well-defined patterns. Examples include:

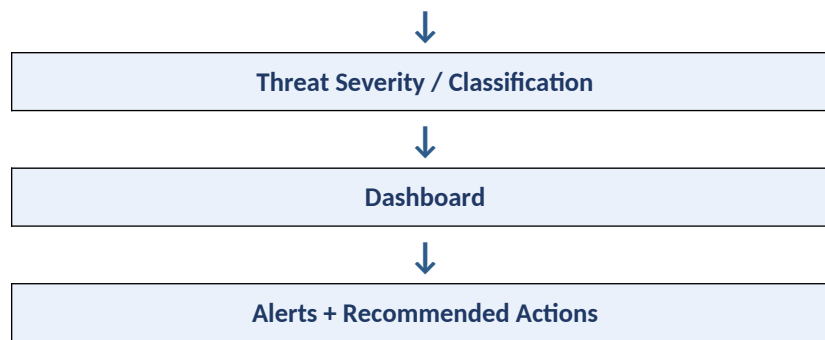
- Repeated failed login attempts → possible brute-force attack.
- Multiple ports probed within a short period → possible port scanning / reconnaissance.
- Unexpected configuration modification → possible configuration tampering.
- Abnormal traffic crossing a defined threshold → possible DoS/DDoS activity.

13. Machine Learning / Anomaly Detection

Machine Learning is used for anomaly detection — identifying behavior that differs significantly from learned or expected normal patterns, including cases that do not exactly match a predefined rule. It is a complement to rule-based detection, not a guarantee of detecting every unknown or zero-day attack.

14. System Workflow





15. Functional Requirements

- Collect logs from IoT devices or a simulated environment.
- Process, filter, and normalize collected logs.
- Apply rule-based detection to identify known patterns.
- Apply ML-based anomaly detection to identify abnormal behavior.
- Identify the affected device for each detected event.
- Classify threat category and assign severity.
- Display relevant evidence and available source information.
- Present results on a centralized dashboard.
- Generate alerts and recommended response actions.

16. Non-Functional Requirements

- Usability: The dashboard should be simple and easy for administrators to understand.
- Performance: Logs and detected events should be processed within a reasonable time.
- Security: Logs and system information should be handled securely.
- Reliability: The system should consistently process logs and produce dependable detections.
- Maintainability: Detection rules and ML models should be easy to update.
- Scalability: The design should reasonably accommodate additional devices and log volume within a lab/demo setting.

17. Technology Stack

Category	Technologies
Core	Python, Scikit-learn, Rule-Based Detection logic, log parsing and data processing
Backend	Flask or FastAPI (one will be selected during design; not both)
Frontend	HTML5, CSS3, JavaScript
Database	SQLite initially; PostgreSQL only if a later need for concurrent/production-scale access arises
Development Tools	Visual Studio Code, Git, GitHub, a modern web browser

React.js, Node.js, and Express.js are intentionally excluded from the current stack; they are not required by this architecture and would add unnecessary complexity for a two-semester student project.

18. System Requirements

Hardware

- Laptop or desktop computer.
- Minimum 8 GB RAM recommended.
- Network connection where required.

Software

- Python and Scikit-learn.
- Flask or FastAPI.
- SQLite (or PostgreSQL if required).
- HTML5, CSS3, JavaScript.
- Visual Studio Code.
- Git and GitHub.
- A modern web browser.

19. Methodology / Implementation Plan

- 1. Requirement Analysis
- 2. System Design
- 3. Log Source Identification
- 4. Log Collection
- 5. Log Processing and Normalization
- 6. Rule-Based Detection
- 7. Machine Learning / Anomaly Detection
- 8. Detection Integration
- 9. Dashboard Development
- 10. Alert and Recommendation System
- 11. Security Event Simulation and Testing
- 12. Performance and Detection Evaluation
- 13. Final Improvements

Testing uses controlled, simulated examples only — including simulated repeated failed logins, simulated port-scanning patterns, simulated abnormal traffic, and simulated configuration changes. No real external systems are targeted.

20. Testing and Evaluation

The system will be evaluated using measurable criteria once implemented. No results are claimed at this stage, as the system has not yet been built.

- Detection accuracy on simulated test events.
- False-positive rate.
- Detection time per event.

- Number of correctly detected test events out of total simulated events.
- System response time (log ingestion to dashboard display).
- Dashboard usability, assessed informally by team/faculty review.

21. Advantages

- Reduces manual effort in reviewing IoT logs.
- Combines rule-based detection with ML-based anomaly detection.
- Provides a centralized view of IoT security events.
- Helps identify the specific device affected by an event.
- Displays severity and relevant evidence to speed up review.
- Recommends response actions rather than leaving administrators to start from scratch.
- Can be extended with additional rules or models over time.

22. Expected Outcome

- Collect relevant IoT logs.
- Process and normalize log data.
- Detect known suspicious activities using rules.
- Detect abnormal behavior using ML.
- Identify affected devices.
- Show threat category and severity.
- Show relevant evidence.
- Show available source information.
- Provide centralized visualization.
- Generate alerts.
- Provide recommended actions.

23. Applications / Use Cases

- Industrial IoT: Monitoring connected industrial devices for suspicious activity.
- Smart Cameras: Monitoring authentication and network activity.
- Smart Locks: Detecting suspicious authentication or unauthorized-access patterns.
- IoT Sensor Networks: Monitoring activity across multiple connected sensors.
- Small/Lab-scale IoT Networks: Giving an administrator centralized visibility into IoT-related security events.

24. Future Scope

The following are potential future enhancements. They are not part of the current build and are listed to show direction, not commitment.

- Real-time log streaming.
- More advanced ML models / deep learning.
- Explainable AI for detection results.
- LLM-based incident summaries.

- Threat intelligence integration.
- SIEM integration.
- Cloud deployment.
- Mobile dashboard.
- Additional IoT protocol support.
- Automated incident response.
- Automated blocking of malicious sources.
- Predictive threat analysis.

25. References

- M. Fagan, K. Megas, K. Scarfone, and M. Smith, "Foundational Cybersecurity Activities for IoT Device Manufacturers," NIST Interagency Report NISTIR 8259, National Institute of Standards and Technology, 2020. Available: <https://doi.org/10.6028/NIST.IR.8259>
- M. Antonakakis et al., "Understanding the Mirai Botnet," in Proc. 26th USENIX Security Symposium, 2017. Available: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis>
- OWASP Foundation, "OWASP Internet of Things Top 10," OWASP IoT Security Project. Available: <https://owasp.org/www-project-internet-of-things/>
- MITRE, "MITRE ATT&CK for Industrial Control Systems (ICS)," The MITRE Corporation. Available: <https://attack.mitre.org/>
- Scikit-learn Developers, "Scikit-learn: Machine Learning in Python — Documentation." Available: <https://scikit-learn.org/stable/documentation.html>
- National Institute of Standards and Technology, "SP 800-213: IoT Device Cybersecurity Guidance for the Federal Government." Available: <https://csrc.nist.gov/pubs/sp/800/213/final>